

Code Audit Report

Bateumz Platform

Full-Stack React + Supabase Application

Raffles / Esports / Live Games / Wallet / Betting / Multi-Region

Date	2026-08-13
Auditor	Z.ai Automated Audit
Scope	Security, Architecture, Performance, Best Practices
Files Scanned	40+ source files, 15 edge functions, 80+ migrations

Table of Contents

1. Executive Summary
2. Project Overview
3. Critical Findings
4. High-Severity Findings
5. Medium-Severity Findings
6. Low-Severity / Best Practices
7. Architecture Assessment
8. Edge Functions Security Review
9. Performance Concerns
10. Recommendations & Roadmap

1. Executive Summary

This audit covers the Bateumz platform, a large-scale React + TypeScript + Vite application backed by Supabase (PostgreSQL, Edge Functions, Auth, Storage, Realtime). The platform combines raffle/lottery management, esports tournaments and leagues, live streaming with interactive games, a digital wallet with deposit/withdrawal capabilities, betting/prediction markets, and multi-region configuration with dynamic branding. The codebase comprises over 200 source files across pages, components, libraries, and server-side edge functions, supported by 80+ database migrations.

The audit identified **4 critical**, **7 high**, **10 medium**, and **8 low-severity** findings. The most concerning issues involve a missing wallet balance check in the spin wheel edge function (enabling free spins), pervasive use of `any` type assertions that disable TypeScript safety across all data access layers, a TOCTOU race condition in the PayPal payment capture flow, and an unprotected IP geolocation endpoint exposing user location data. On the positive side, the platform demonstrates strong auth error handling with anti-enumeration protection, well-designed server-authoritative payment flows, and robust error boundaries for the game loading system.

Severity	Count	Categories
CRITICAL	4	Missing balance check, Type safety bypass, Race condition, Data exposure
HIGH	7	Auth bypass risk, CORS wildcard, Unsafe casts, Missing input validation
MEDIUM	10	Silent error swallowing, Hardcoded strings, Memory leaks, Code duplication
LOW	8	Naming, Dead code, Bundle size, Minor improvements

Table 1: Audit findings summary by severity level

2. Project Overview

2.1 Technology Stack

The application is built as a single-page application using React 18 with TypeScript, bundled via Vite. Routing is handled by React Router v6 with Framer Motion page transitions. State management relies on React Context API (Auth, Language, Theme, Currency, Region, Mobile Navigation, Dynamic Theme, Regional Theme) supplemented by TanStack React Query for server state. The UI framework is built on shadcn/ui components with Tailwind CSS for styling. Payment processing integrates PayPal (client SDK + server-side order/capture) and Stripe (via Edge Functions). The backend is entirely Supabase: PostgreSQL database, Deno-based Edge Functions, Row Level Security policies,

and Realtime subscriptions for live features.

Layer	Technology	Notes
Frontend	React 18 + TypeScript + Vite	SPA with Framer Motion transitions
UI	shadcn/ui + Tailwind CSS	50+ UI primitives, custom themes
State	Context API + React Query	8 context providers in App.tsx
Backend	Supabase (PostgreSQL + Deno Edge)	15+ edge functions, 80+ migrations
Auth	Supabase Auth	Email/password + OAuth, RLS policies
Payments	PayPal + Stripe	Server-authoritative pricing
Realtime	Supabase Realtime	Live games, chat, draw events

Table 2: Technology stack overview

2.2 Module Breakdown

The platform is organized into several major feature domains, each with dedicated pages, components, and server-side logic. The Raffle module handles prize draw management with PayPal/Stripe payment integration, ticket generation, live draw streaming, and draw result verification. The Esports module provides tournament bracket management, season/league systems with round-robin scheduling, team management, player transfers, a betting and prediction market system, anti-cheat reporting, and an achievements/reputation framework. The Live Games module is the largest, containing over 70 interactive mini-games (Tic-Tac-Toe VS, Chess, Snake Battle, Quiz Battle, Bingo, Millionaire, and many more) that can be played during live streams with real-time multiplayer support. The Wallet module handles balance management, deposits, withdrawals, and transaction history across multiple payment methods including M-Pesa, PIX, bank transfer, and cryptocurrency.

Additional modules include a Blog/CMS system with SEO support, a gamification engine (XP, levels, streaks, achievements), an ambassador/referral program with luck points, a multi-region configuration system allowing different countries to have independent branding, settings, and feature flags, and a comprehensive admin panel with user management, revenue analytics, audit logging, cron job scheduling, and voucher management. The application also supports PWA installation, push notifications, and has overlay modes for live stream integration with OBS Studio.

3. Critical Findings

3.1 [C-01] Missing Wallet Balance Check in Spin Wheel Edge Function

The spin-wheel-spin Edge Function does not verify or deduct the user's wallet balance before recording a spin session. Although the database may define a `spin_cost` field on the wheel game record, the function reads it into a local variable but never uses it to charge the user. This means any authenticated user can spin the wheel an unlimited number of times without paying, potentially winning real prizes or reward points. The function correctly validates the user's authentication token and checks prize win limits per segment, but the fundamental economic gate, which is the balance deduction, is entirely absent.

Impact: Financial loss to the platform. Users can drain prize pools and accumulate rewards without any cost. In a production environment with real-money prizes, this represents a direct revenue leak that could be exploited automatically at scale.

Recommendation: Before inserting the spin session record, call an RPC function (e.g., `wallet_process_transaction`) to debit the user's wallet by the wheel's `spin_cost` amount. Wrap the balance check and session insertion in a database transaction to prevent TOCTOU race conditions. The function should return an error response if the user's balance is insufficient, similar to how the battle system checks balance before creating a battle in `battles.ts`.

3.2 [C-02] Pervasive Type Safety Bypass via 'as any' Casts

Nearly every data access layer file in the project casts the Supabase client to any at the module level using `const sb: any = supabase`. This pattern appears in `wallet.ts`, `stripe.ts`, `esports.ts`, `esports-advanced.ts`, `battles.ts`, `gamification.ts`, and `audit.ts`. While the project generates TypeScript types via `supabase gen-types` (present in `integrations/supabase/types.ts`), these types are never actually used for client queries. The Database generic parameter is correctly passed to `createClient` in `client.ts`, but every downstream consumer immediately discards this type safety.

Impact: This completely negates the value of TypeScript in the data access layer. Any column rename, type change, or schema migration will not produce compile-time errors, only runtime failures. Given that the project has 80+ migrations with frequent schema changes, this is a significant maintainability and reliability risk. Additionally, many function calls use `as any` on insert data (e.g., `audit.ts` line 37-43, `AuthContext.tsx` line 60, `gamification.ts` line 138), which bypasses even basic structural validation.

Recommendation: Remove all `const sb: any = supabase` aliases and use the typed client directly. For table inserts that don't match the generated types, create proper interface definitions that extend the generated types. This is a significant refactor but essential for a codebase of this size and complexity.

3.3 [C-03] TOCTOU Race Condition in PayPal Capture

The paypal-capture-order Edge Function checks if an order has already been processed by querying the participants table for the PayPal order ID, but this check and the subsequent insert are not wrapped in a database transaction. Between the SELECT count and the INSERT, a concurrent request could pass the same check and both requests would insert tickets, resulting in duplicate ticket creation for a single payment. The replay protection at line 72-78 of the edge function checks for existing records with the same paypal_order_id, but without a serializable transaction or a unique constraint on that column, this is vulnerable to timing attacks.

Impact: A user could potentially get double the tickets for a single PayPal payment by sending two concurrent capture requests. The financial loss depends on ticket price and prize value. While PayPal itself prevents double-charging the buyer, the platform would issue double the tickets.

Recommendation: Add a UNIQUE constraint on participants.paypal_order_id in the database. Alternatively, wrap the check-and-insert logic in a SERIALIZABLE transaction or use an INSERT ... ON CONFLICT (upsert) pattern. The unique constraint approach is simplest and most reliable as it provides database-level protection regardless of application logic.

3.4 [C-04] Unprotected IP Geolocation Endpoint

The detectUserRegion function in useRegionalConfig.tsx (line 97) calls the external API <https://ipwho.is/> directly from the browser to determine the user's country. This exposes the user's IP address to a third-party service without any disclosure or consent. Furthermore, the response from this endpoint is used to set regional configuration including available features, currency, and branding. A malicious actor could intercept or manipulate this response (via DNS spoofing or MITM on the unencrypted endpoint) to force a different region configuration, potentially enabling features or pricing that should not be available in the user's actual region.

Impact: Privacy violation (user IP sent to third party without consent) and potential configuration manipulation. If regional pricing differs, this could lead to price arbitrage or access to region-locked features.

Recommendation: Move geolocation to a server-side Edge Function that proxies the request through Supabase. This hides the user's IP from third parties and allows server-side validation and caching of results. Add a privacy policy disclosure about IP-based region detection. Consider using Supabase's built-in request headers or a self-hosted GeoIP database (e.g., MaxMind) to avoid external API dependencies entirely.

4. High-Severity Findings

4.1 [H-01] ProtectedRoute Missing 'superadmin' Role Check

The ProtectedRoute component handles 'admin', 'business', and 'regional_manager' roles but never explicitly validates the 'superadmin' role. The admin check (line 29) allows both 'admin' AND 'superadmin' through, which is correct for admin routes. However, there is no route that is exclusively restricted to 'superadmin' users, meaning superadmin-only operations (like the Super Dashboard) have the same access control as regular admin operations. If the system is designed to have a hierarchy where superadmins have exclusive access to certain features, this hierarchy is not enforced at the route level.

Recommendation: Add an optional superadmin role check and use it for routes that require superadmin privileges exclusively.

4.2 [H-02] Wildcard CORS in Edge Functions

Multiple Edge Functions define CORS headers with Access-Control-Allow-Origin: *. While the functions do validate the Authorization Bearer token, the wildcard CORS header allows any website to make authenticated cross-origin requests to these functions if the user's token is leaked via XSS or a malicious extension. Functions affected include spin-wheel-spin and potentially others using the same CORS pattern.

Recommendation: Restrict CORS origins to the actual platform domains. Use environment variables to configure allowed origins and validate the request Origin header against the allowlist.

4.3 [H-03] PayPal getAccessToken Missing Error Check

In both paypal-create-order and paypal-capture-order Edge Functions, the getAccessToken helper does not check whether the response JSON actually contains an access_token before returning it. If the PayPal API returns an error response (invalid credentials, service outage), the function returns undefined, which then causes an unhelpful 'Authorization header must be Bearer undefined' error from PayPal's order endpoints.

Recommendation: Add a null check after parsing the token response and throw a descriptive error if access_token is missing.

4.4 [H-04] Client-Side Spin Cost Not Verified Server-Side for Battles

The createBattle function in battles.ts checks the user's balance client-side before calling the server RPC, but the server-side RPC function is not visible in the codebase. If the RPC

does not independently verify the balance, a user could bypass the client check by calling the RPC directly with arbitrary parameters via the Supabase client.

Recommendation: Audit the `create_battle` RPC to ensure it performs its own balance check and deduction within a transaction.

4.5 [H-05] No Rate Limiting on Expensive Operations

Several Edge Functions and client-side operations (spin wheel, battle creation, bet placement, deposit requests) lack rate limiting. A malicious user could call these endpoints in rapid succession, potentially causing financial loss (spin wheel without balance check) or database load (creating hundreds of battles).

Recommendation: Implement per-user rate limiting using Supabase's `pg_net` or a Redis layer. Use a sliding window approach with reasonable limits per operation type.

4.6 [H-06] AuthContext Referral Processing Has No Server Validation

The referral processing in `AuthContext.tsx` (lines 122-147) reads a referral code from `localStorage` and directly inserts referral records and awards luck points entirely client-side. A malicious user could manipulate `localStorage` to claim referral bonuses from arbitrary users or create self-referential loops by setting the ref code before signup.

Recommendation: Move referral processing to a server-side function or database trigger. Validate that the referrer exists, is not the same as the referred user, and has not exceeded referral limits.

4.7 [H-07] Missing Input Validation on Betting Page

The `BettingPage.tsx` (esports section) has a local `useUserId` hook that silently swallows errors (line 84: empty catch). The page processes real-money bets but the user ID fetching has no retry logic or error state, potentially allowing bets to be placed with a null user ID if the auth state is inconsistent.

Recommendation: Add proper error handling with user-facing feedback. Never allow bet placement if user ID is null or loading.

5. Medium-Severity Findings

5.1 [M-01] Pervasive Silent Error Swallowing

Empty catch blocks appear throughout the codebase. AuthContext.tsx lines 118 and 146 have bare `catch() {}` blocks that swallow errors during profile updates and referral processing. Gamification.ts uses `.catch(() => {})` on RPC calls and achievement inserts. While some of these may be intentionally non-fatal, they make debugging extremely difficult and can mask real issues in production. At minimum, these should log to an error tracking service.

Recommendation: Add structured error logging (e.g., Sentry integration) to all catch blocks. Even for non-fatal operations, log the error context for debugging.

5.2 [M-02] Hardcoded Portuguese Strings in Game Components

The SafeGameLoader component and many game components contain hardcoded Portuguese strings (e.g., 'Erro ao carregar', 'Tentando novamente', 'Outro jogo') instead of using the `i18n` system. Given that the LanguageContext supports 6 locales (en, pt, pt-BR, es, fr, hi), these hardcoded strings will not be translated for international users.

Recommendation: Use the LanguageContext `t()` function for all user-facing strings in game components.

5.3 [M-03] AuthProvider nesting 8 layers deep

App.tsx wraps the application in 8 nested context providers (QueryClient, Helmet, RegionalConfig, Theme, DynamicTheme, Language, Currency, RegionalTheme, PayPal, Auth). Each provider adds render overhead and makes the component tree harder to debug. If any provider's state changes, all children re-render.

Recommendation: Consider using a component composition pattern or a provider aggregator to reduce nesting. Evaluate which providers actually need to wrap the entire app vs. only specific route subtrees.

5.4 [M-04] Duplicate Loading Screens with Different Timings

main.tsx shows a LoadingScreen for 2200ms, then App.tsx shows another for 1800-4000ms. Users see a loading screen for potentially 6.2 seconds total. Neither loading screen shows actual progress or indicates what is being loaded.

Recommendation: Remove the redundant loading screen in main.tsx. Use a single loading state that tracks actual initialization progress rather than arbitrary timeouts.

5.5 [M-05] LanguageContext.tsx is 106KB

The `LanguageContext` file contains all translations for 6 languages inline, making it over 106KB. This entire file is parsed and loaded on every page load, even though most pages only use a fraction of the translations. This impacts initial bundle size and parse time.

Recommendation: Split translations into separate locale files (e.g., `locales/en.json`, `locales/pt.json`) and lazy-load them based on the active language. Use dynamic imports to reduce the initial bundle.

5.6 [M-06] No Debouncing on Live Game State Updates

Live game components send state updates (scores, chat messages, reactions) without debouncing. In fast-paced games with many concurrent players, this could generate thousands of Supabase Realtime messages per second, potentially overwhelming the database and WebSocket connections.

Recommendation: Implement debouncing and batching for high-frequency state updates. Use local state for intermediate updates and sync to the server at a controlled interval (e.g., 100ms for game state, 500ms for analytics).

5.7 [M-07] Wallet Transaction Race Condition in Withdrawal

The `createWithdrawalRequest` function in `wallet.ts` first checks the balance, then processes a debit transaction, then creates the withdrawal record as three separate database calls. Between the balance check and the debit, another concurrent withdrawal could pass the same balance check, resulting in over-withdrawal.

Recommendation: Use a database RPC that wraps the balance check and debit in a single transaction with a `SERIALIZABLE` isolation level.

5.8 [M-08] Exposed Error Details in Edge Functions

Several Edge Functions return raw error messages to the client via `String(e)`. This can expose internal implementation details, database schema information, or stack traces to end users. While this is useful during development, it should be disabled in production.

Recommendation: Return generic error messages to clients and log detailed errors server-side. Use environment-based error detail levels (verbose in development, minimal in production).

5.9 [M-09] Missing Pagination on Several Data Fetches

Functions like `getMyBattles` (limit 30), `getTransactions` (limit 30), and several admin data fetches use simple limits without cursor-based or offset-based pagination. For admin users managing large datasets, this means they cannot access data beyond the first page.

Recommendation: Implement proper pagination with cursor-based approach for large datasets and infinite scroll for user-facing lists.

5.10 [M-10] No Content Security Policy Headers

The application does not implement Content Security Policy (CSP) headers, which help prevent XSS attacks by restricting which scripts, styles, and resources can be loaded. Given the application integrates third-party payment SDKs (PayPal, Stripe) and loads external resources, CSP is important for defense in depth.

Recommendation: Add CSP headers via the Supabase configuration or a reverse proxy. Start with a restrictive policy and use report-only mode to identify necessary exceptions before enforcement.

6. Low-Severity / Best Practices

6.1 [L-01] Inconsistent Currency Formatting

wallet.ts hardcodes MZN (Mozambican Metical) in formatMZN(), but the platform supports multiple currencies via the CurrencyContext. The esports-advanced.ts and betting system use different currency handling. This creates inconsistency across the platform when displaying monetary values.

Recommendation: Create a single formatting utility that uses the active currency from CurrencyContext and supports all platform currencies.

6.2 [L-02] Unused Import in App.tsx

App.tsx imports KahootMultiplayerQuiz, LiveBingo, and ChallengeRoulette components but they are not used in any route. These lazy-loaded components may still be included in the bundle depending on the bundler configuration.

Recommendation: Remove unused imports to reduce bundle size and improve code clarity.

6.3 [L-03] Mixed Language in Code Comments

Code comments alternate between Portuguese, English, and sometimes both in the same file. This makes the codebase harder to maintain for international development teams. Pick one language and be consistent.

Recommendation: Standardize on English for all code comments and commit messages, with Portuguese reserved for user-facing strings.

6.4 [L-04] No Test Coverage for Critical Paths

The project has a basic test setup (`vitest.config.ts`, `test/setup.ts`, `example.test.ts`) but the example test is a placeholder. There are no tests for payment flows, wallet operations, auth flows, or game logic.

Recommendation: Prioritize tests for: payment capture, wallet balance operations, spin wheel with balance check, and battle settlement.

6.5 [L-05] Large Game Component Files

Many game components under `components/livegames/` are likely large single-file components. Combined with the fact that there are over 70 games, this creates a massive codebase that is difficult to maintain and could benefit from code splitting and shared game utilities.

Recommendation: Extract common game logic (score tracking, turn management, win conditions) into shared hooks or utility modules.

6.6 [L-06] Missing Error Boundary for Payment Components

`PayPalCheckout` and `StripeCheckout` components do not have individual error boundaries. A payment SDK failure could crash the entire page rather than showing a graceful fallback.

Recommendation: Wrap payment components in error boundaries with user-friendly fallback UI and retry options.

6.7 [L-07] No Request Cancellation on Unmount

Many `useEffect` hooks in pages and components initiate Supabase queries but do not return cleanup functions to cancel pending requests when the component unmounts. This can cause state updates on unmounted components and memory leaks.

Recommendation: Use `AbortController` for fetch calls and Supabase's `.abortController()` for real-time subscriptions in `useEffect` cleanup.

6.8 [L-08] Potential Memory Leak in Auth Context

The `onAuthStateChange` callback in `AuthContext.tsx` uses a `setTimeout` wrapper (line 172) to defer state updates. If the component unmounts between the timeout scheduling and execution, the state update will attempt to modify unmounted component state. While the 'active' flag partially mitigates this, the timeout reference is not cleaned up.

Recommendation: Store the timeout reference and clear it in the `useEffect` cleanup function alongside the subscription `unsubscribe`.

7. Architecture Assessment

7.1 What Works Well

The platform demonstrates several commendable architectural decisions. The server-authoritative payment flow is well-designed: the PayPal create-order function fetches ticket prices server-side (preventing price tampering), and the capture function verifies the paid amount matches expected tickets before inserting records. The auth error handling system in `authErrors.ts` implements proper anti-enumeration, never revealing whether an email exists in the system. The `SafeGameLoader` provides excellent resilience for the game loading system with error boundaries, retry mechanisms, and user-friendly fallback UI. The multi-region configuration system is sophisticated, allowing independent branding, feature flags, and settings per country. The audit logging system provides accountability for administrative actions.

7.2 Architectural Concerns

The monolithic `App.tsx` imports over 130 page components eagerly, which will significantly impact initial load time. While Vite may code-split these into separate chunks, the route definitions themselves create tight coupling. The 8 nested context providers create a deep render tree where any state change in an outer provider triggers re-renders of all inner providers and their children. The `lib/` directory contains a mix of pure utility functions and direct Supabase query functions without a clear separation, making it hard to test business logic in isolation. The 80+ database migrations suggest rapid schema evolution without a clear migration strategy, which increases the risk of schema drift between environments.

7.3 Scalability Considerations

The platform's architecture has several scalability bottlenecks that should be addressed before user growth. The Supabase Realtime system is used for live games, chat, and notifications, but there is no visible strategy for managing connection counts or channel isolation. With 70+ game types and potentially thousands of concurrent players, the Realtime layer could become a bottleneck. The edge functions use per-request Supabase client creation in some cases (spin-wheel-spin creates two clients per request), which adds latency. The wallet system uses optimistic client-side operations without proper server-side locking, which will fail under concurrent access. Consider implementing connection pooling for edge functions, channel namespace isolation for Realtime, and

server-side wallet operations within database transactions for data integrity under load.

8. Edge Functions Security Review

The platform uses 15+ Supabase Edge Functions written in Deno/TypeScript for server-side operations including payment processing, game logic, notifications, and administrative tasks. The following table summarizes the security assessment of the key edge functions reviewed during this audit.

Function	Auth	Input Validation	Issues
paypal-create-order	Yes	Good	Missing token null check
paypal-capture-order	Yes	Good	TOCTOU race condition
spin-wheel-spin	Yes	Partial	No balance check
paypal-config	No	N/A	Exposes client ID publicly (by design)
auth-email-hook	Service	Unknown	Not reviewed in detail
auto-draw	Service	Unknown	Not reviewed in detail

Table 3: Edge functions security assessment summary

9. Performance Concerns

9.1 Bundle Size

With over 130 eagerly imported page components, 70+ game components, and 50+ UI primitives, the application's JavaScript bundle is likely very large. While Vite performs code splitting for dynamic imports, the static imports in App.tsx and the 106KB LanguageContext file will be in the initial bundle. The Framer Motion library adds significant weight, and the PayPal/Stripe SDKs are loaded for all users regardless of whether they visit a payment page. The 8 nested context providers mean that every state change in an outer provider triggers reconciliation of the entire component tree below it. For a platform targeting mobile users in regions with slow connections (Mozambique, Brazil), bundle size optimization should be a high priority.

9.2 Database Query Patterns

Several lib/ functions make multiple sequential database calls that could be parallelized. For example, getUserGamification in gamification.ts correctly uses Promise.all for three parallel fetches, but other functions like fetchProfile in AuthContext.tsx make 3-4

sequential awaits that could run concurrently. The `createWithdrawalRequest` function makes 3 sequential calls (`getBalance`, `processTransaction`, `insert withdrawal`) when the first two should be atomic. The `esports-advanced.ts` file (1776 lines) contains many complex query patterns that would benefit from batching and caching.

9.3 Realtime Scaling

The live game system relies heavily on Supabase Realtime for multiplayer game state synchronization. Each game session likely creates Realtime channels for game state, chat, and reactions. With 70+ game types and the potential for hundreds of concurrent game sessions during live events, the Realtime connection count could become a bottleneck. There is no visible strategy for channel cleanup after games end, connection pooling, or message throttling. Consider implementing a game session lifecycle manager that ensures Realtime channels are properly cleaned up when games end, and batch state updates to reduce message frequency.

10. Recommendations and Roadmap

10.1 Immediate Actions (Week 1-2)

- Fix C-01: Add wallet balance check and deduction to spin-wheel-spin Edge Function.
- Fix C-03: Add UNIQUE constraint on `participants.paypal_order_id` column.
- Fix H-06: Move referral processing from `AuthContext` to a server-side function.
- Fix C-04: Move IP geolocation from client to server-side Edge Function.
- Fix H-03: Add null check on `PayPal access_token` in `getAccessToken` helper.

10.2 Short-Term (Month 1)

- Begin C-02 remediation: Start removing 'as any' casts from the most critical paths (wallet, payments, bets).
- Implement rate limiting on all Edge Functions that handle financial operations.
- Add CSP headers and restrict CORS to platform domains only.
- Split `LanguageContext` translations into separate locale files with lazy loading.
- Remove duplicate loading screens and implement single progress-tracked loading state.
- Add error boundaries around payment components and implement request cancellation in `useEffect` cleanups.

10.3 Medium-Term (Month 2-3)

- Complete type safety remediation across all lib/ files.
- Implement proper pagination with cursor-based approach for all list endpoints.
- Add unit and integration tests for payment flows, wallet operations, and game logic.
- Refactor context provider nesting using composition patterns.
- Implement Realtime channel lifecycle management for live games.
- Move all game string literals to the i18n translation system.

10.4 Long-Term (Quarter 2+)

- Implement comprehensive E2E test coverage using Playwright (already configured).
- Migrate to a proper state management solution (Zustand or Jotai) to reduce context re-renders.
- Implement micro-frontend architecture to allow independent deployment of feature modules.
- Add observability stack (Sentry for errors, DataDog for performance, PostHog for analytics).
- Conduct penetration testing with a focus on the wallet and payment systems.
- Implement database connection pooling and read replicas for scaling.